

Atividade Prática 8 – Máquina de Estados

Pedro Lucas dos Reis Silva - RA: 2272040

Lucas dos Reis Viana - RA: 2321645

11 de junho de 2026

1 Introdução

Esta atividade consiste na implementação, em VHDL, de um controlador para quatro LEDs de um acessório luminoso com dois chifres. O circuito possui dois LEDs do lado direito, R1 e R2, e dois LEDs do lado esquerdo, L1 e L2. A proposta é controlar esses LEDs por meio de modos de operação selecionados por botão, com opção de pausa, reset e duas velocidades de animação.

Os conceitos novos utilizados foram:

- **Máquina de estados:** estrutura usada para alternar entre os modos ACESO, INTERMITENTE, ALTERNADO e SEQUENCIAL;
- **Máquina de Moore:** as saídas dos LEDs dependem do estado atual e do subestado da animação;
- **Deteção de borda:** recurso usado para que o botão de troca de modo avance apenas uma vez por pressionamento;
- **Temporização síncrona:** contador acionado pelo clock de 50 MHz para controlar a velocidade dos modos animados.

2 Desenvolvimento

2.1 Arquivos do Projeto

O projeto foi organizado nos seguintes arquivos:

- `corsimcornao.vhd`: contém a entidade principal e a lógica da máquina de estados;
- `atividade8.qpf`: arquivo de projeto do Quartus Prime;
- `atividade8.qsf`: arquivo de configurações do Quartus, incluindo o dispositivo, a entidade de topo, o arquivo VHDL e a pinagem da placa.

A entidade principal do projeto é `corsimcornao`. O projeto utiliza o dispositivo 10M50DAF484C7G, da família MAX 10, presente na placa DE10-Lite.

2.2 Entradas e Saídas

A entidade `corsimcornao` possui as seguintes portas:

- `clk`: entrada de clock de 50 MHz da placa;

- `rst_n`: entrada de reset assíncrono, ativa em nível baixo, conectada ao botão KEY0;
- `mode_btn_n`: entrada de troca de modo, ativa em nível baixo, conectada ao botão KEY1;
- `enable`: entrada de habilitação, conectada à chave SW0;
- `speed_sel`: entrada de seleção de velocidade, conectada à chave SW1;
- `leds`: saída de 4 bits conectada aos LEDs LEDR0 a LEDR3.

O vetor `leds` foi associado aos LEDs do acessório na ordem `leds(0)=R1`, `leds(1)=R2`, `leds(2)=L1` e `leds(3)=L2`.

2.3 Funcionamento Lógico

A máquina principal possui quatro estados. No estado **ACESO**, todos os LEDs permanecem ligados. No estado **INTERMITENTE**, os LEDs internos de cada chifre alternam entre os padrões 0101 e 1010. No estado **ALTERNADO**, os lados direito e esquerdo piscam alternadamente com os padrões 0011 e 1100. No estado **SEQUENCIAL**, apenas um LED fica aceso por vez, seguindo a ordem R1, R2, L1 e L2.

O botão KEY1 é ativo em nível baixo, por isso a troca de modo ocorre na borda de descida do sinal `mode_btn_n`. O sinal anterior do botão é armazenado em `mode_btn_reg`, permitindo identificar a transição de '1' para '0'.

A animação é controlada pelo sinal `sub_estado`, que varia de 0 a 3. Esse subestado é incrementado pelo temporizador quando `enable` está em nível alto. Quando `enable` está em nível baixo, o temporizador deixa de avançar e o circuito permanece pausado no padrão atual.

A chave `speed_sel` define o limite do temporizador. Com `speed_sel = '0'`, o limite é $F_{CLK}/2$, resultando em uma troca aproximadamente a cada 0,5 s. Com `speed_sel = '1'`, o limite é $F_{CLK}/10$, resultando em uma troca aproximadamente a cada 0,1 s.

2.4 Implementação na Placa DE10-Lite

Na implementação física, o circuito utiliza o clock de 50 MHz, os botões KEY0 e KEY1, as chaves SW0 e SW1 e os LEDs LEDR0 a LEDR3. A Tabela 1 apresenta as atribuições definidas no arquivo `atividade8.qsf`.

Tabela 1: Pinagem da máquina de estados na placa DE10-Lite.

Recurso	Sinal VHDL	Função	Pino
MAX10_CLK1_50	<code>clk</code>	clock de 50 MHz	PIN_P11
KEY0	<code>rst_n</code>	reset assíncrono	PIN_B8
KEY1	<code>mode_btn_n</code>	troca de modo	PIN_A7
SW0	<code>enable</code>	habilitação e pausa	PIN_C10
SW1	<code>speed_sel</code>	seleção de velocidade	PIN_C11
LEDR0	<code>leds(0)</code>	LED R1	PIN_A8
LEDR1	<code>leds(1)</code>	LED R2	PIN_A9
LEDR2	<code>leds(2)</code>	LED L1	PIN_A10
LEDR3	<code>leds(3)</code>	LED L2	PIN_B10

3 Simulação

A simulação gate-level apresenta o comportamento dos sinais `clk`, `rst_n`, `mode_btn_n`, `enable`, `speed_sel` e `leds`. O reset leva o circuito ao modo ACESO, e os acionamentos do botão percorrem os estados INTERMITENTE, ALTERNADO, SEQUENCIAL e novamente ACESO. Na saída, são observados os padrões 1111, 0101, 0011, 0001 e 1111, confirmando as transições da máquina de estados.

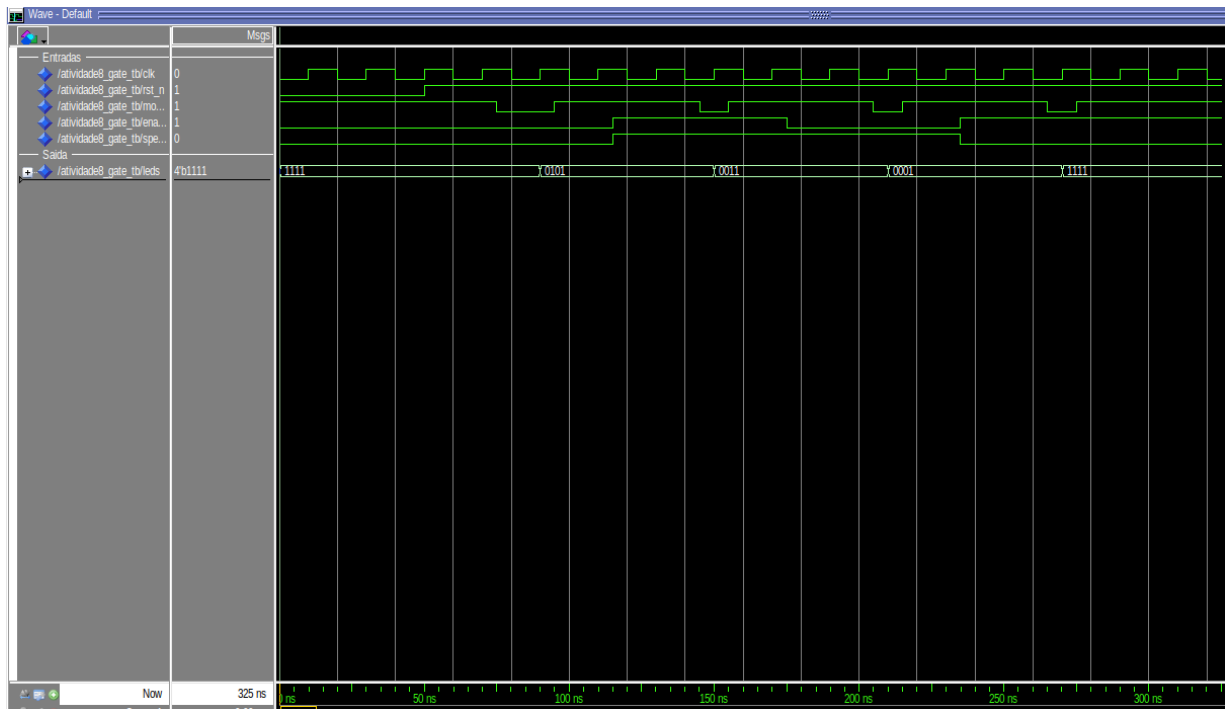


Figura 1: Simulação gate-level da máquina de estados `corsimcornao`.

4 Diagrama RTL

O diagrama RTL gerado pelo Quartus Prime representa a entidade `corsimcornao` e a lógica sequencial responsável pelo registro do modo atual, pelo temporizador, pelo subestado da animação e pela detecção de borda do botão. A lógica combinacional associada seleciona o próximo modo de operação e define o padrão de saída aplicado ao vetor `leds`.

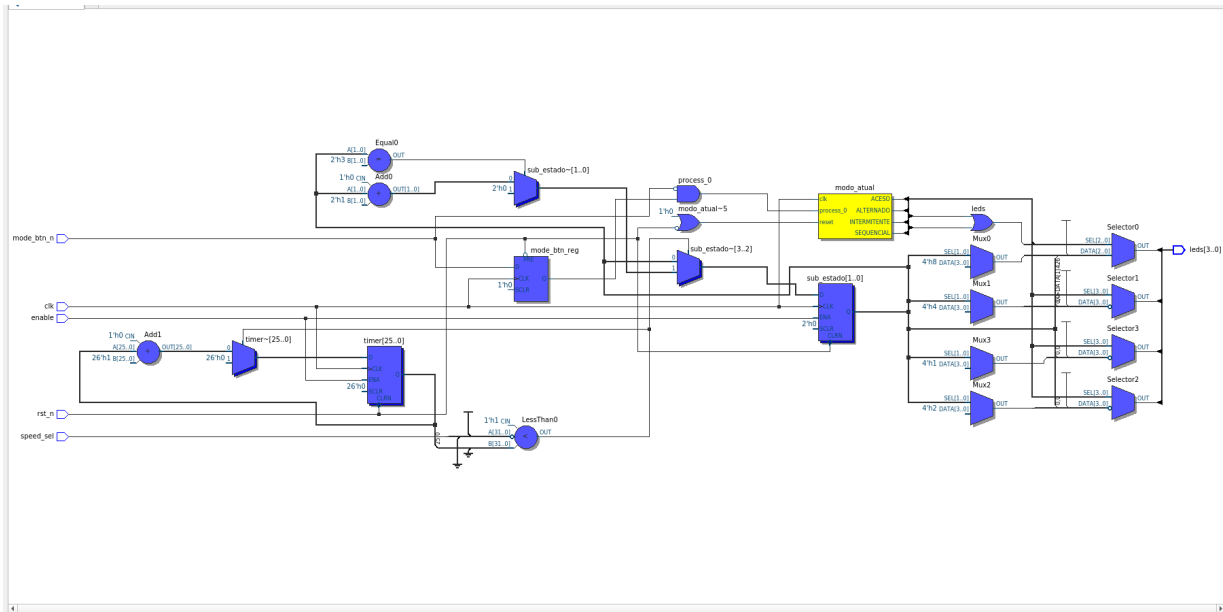


Figura 2: Diagrama RTL da máquina de estados corsimcornao.

5 Considerações Finais

A atividade permitiu implementar um controlador de LEDs baseado em máquina de estados finita. A solução alterna entre quatro modos de operação, utiliza um botão para mudança de modo, possui reset ativo em nível baixo, pausa por **enable** e duas velocidades selecionáveis por chave. O circuito foi descrito de forma síncrona, com separação entre registradores de estado, temporização e lógica de saída.

A Código VHDL – Entidade corsimcornao

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity corsimcornao is
6     generic (
7         F_CLK : integer := 50_000_000
8     );
9     port (
10         clk          : in  std_logic;
11         rst_n         : in  std_logic;
12         mode_btn_n    : in  std_logic;
13         enable        : in  std_logic;
14         speed_sel     : in  std_logic;
15         leds          : out std_logic_vector(3 downto 0)
16     );
17 end entity;
18
19 architecture comportamento of corsimcornao is
20     type modo_type is (ACESO, INTERMITENTE, ALTERNADO, SEQUENCIAL);
21
22     signal modo_atual    : modo_type;
23     signal proximo_mod0 : modo_type;
24     signal timer         : integer range 0 to F_CLK := 0;
25     signal limite_timer  : integer;
26     signal mode_btn_reg  : std_logic;
27     signal sub_estado    : integer range 0 to 3 := 0;
28 begin
29     limite_timer <= F_CLK / 2 when speed_sel = '0' else F_CLK / 10;
30
31     process(clk, rst_n)
32     begin
33         if rst_n = '0' then
34             modo_atual <= ACESO;
35             mode_btn_reg <= '1';
36         elsif rising_edge(clk) then
37             mode_btn_reg <= mode_btn_n;
38
39             if mode_btn_reg = '1' and mode_btn_n = '0' then
40                 modo_atual <= proximo_mod0;
41             end if;
42         end if;
43     end process;
44
45     process(modo_atual)
46     begin
47         case modo_atual is
48             when ACESO => proximo_mod0 <= INTERMITENTE;
49             when INTERMITENTE => proximo_mod0 <= ALTERNADO;

```

```

50         when ALTERNADO      => proximo_modos <= SEQUENCIAL;
51         when SEQUENCIAL      => proximo_modos <= ACESO;
52     end case;
53 end process;
54
55 process(clk, rst_n)
56 begin
57     if rst_n = '0' then
58         timer <= 0;
59         sub_estado <= 0;
60     elsif rising_edge(clk) then
61         if enable = '1' then
62             if timer >= limite_timer then
63                 timer <= 0;
64
65                 if sub_estado = 3 then
66                     sub_estado <= 0;
67                 else
68                     sub_estado <= sub_estado + 1;
69                 end if;
70             else
71                 timer <= timer + 1;
72             end if;
73         end if;
74     end if;
75 end process;
76
77 process(modos_atual, sub_estado)
78 begin
79     case modos_atual is
80         when ACESO =>
81             leds <= "1111";
82
83         when INTERMITENTE =>
84             if sub_estado mod 2 = 0 then
85                 leds <= "0101";
86             else
87                 leds <= "1010";
88             end if;
89
90         when ALTERNADO =>
91             if sub_estado mod 2 = 0 then
92                 leds <= "0011";
93             else
94                 leds <= "1100";
95             end if;
96
97         when SEQUENCIAL =>
98             case sub_estado is
99                 when 0      => leds <= "0001";
100                when 1      => leds <= "0010";

```

```

101         when 2      => leds <= "0100";
102         when 3      => leds <= "1000";
103         when others => leds <= "0000";
104     end case;
105 end case;
106 end process;
107 end architecture;

```

Listing 1: Arquivo corsimcornao.vhd.